What we built

Three pieces, working together:

1. `CLAUDE-CCDS.md` — a conventions file that tells an agent *how to write code* in a ccds-structured repo, day to day (not just at scaffold time): directory layout, config/CLI conventions, `Protocol`-based design, testing, tooling.
2. `new-ds-project` **skill** — scaffolds a **brand-new** repo into this structure from scratch.
3. `refactor-ccds` **skill** — takes an **existing**, non-ccds repo and reorganizes it into our style, rather than moving files ad hoc.

The two skills handle one-time setup/migration; the `.md` file keeps every later change consistent with it.

ccds file structure

```
data/
    raw/ # immutable, gitignored, never edited in place
    interim/ # regenerable from raw
    processed/ # regenerable, model-ready
notebooks/ # exploration only: <n>.<initials>-<desc>.ipynb
models/ # serialized trained models
reports/
    figures/ # generated charts
references/ # data dictionaries, explanatory material
docs/adr/ # architecture decision records
src/<module_name>/
    config.py # paths, env, logging, seeding
    data/ make_dataset.py, validators.py
    features/* base.py, build_features.py
    modeling/ base.py, train.py, predict.py, architectures.py
    visualization/ plots.py
tests/ # mirrors src/<module_name>/, one test file per source file
```

Tools we're using

Config & CLI

- `pathlib` + `config.py`
- `python-dotenv`
- `typer` (simple stages)
- `Tyro` (config-heavy stages)
- `loguru` + `tqdm`

Data & experiments

- `pandera` (schema validation)
- `wandb` (run tracking)
- `Makefile` (orchestration)

Quality

- `ruff` (lint + format)
- `mypy` (typing)
- `pytest` (tests)
- `pre-commit` (blocks commits)
- `pip-audit` (dependency security)

Method structure as `Protocols`

- Each stage's method structure is broken into a `typing.Protocol` interface with swappable concrete implementations — decomposed into the *real* sub-parts rather than one interface per stage:

```
class FeatureExtractor(Protocol): def transform(self, raw): ...
class Backbone(Protocol): def encode(self, features): ...
class PredictorHead(Protocol): def forward(self, embedding): ...
class Model(Protocol):
    def fit(self, features, labels) -> None: ...
    def predict(self, features): ...
```

- This decomposition also gives us something to **visualize**: diagram the protocols and their information flow/containment (what calls what, what wraps what) to see a program's high-level structure and building blocks at a glance — independent of any one implementation's internals.

Style & workflow — the small things

- **Style**: `ruff` enforces PEP-8-ish formatting + linting, roughly followed rather than dogmatically; NumPy-style docstrings on public functions/classes; comments explain *why*, not *what*; no commented-out code or context-free TODOs.
- **Git/GitHub**:
  - Commit early and often, imperative-mood messages explaining *why*.
  - Branch for anything experimental/risky; merge or discard.
  - Push regularly — it's the backup and shareable record.
  - Never force-push or rewrite history on `main`.
  - No PR requirement or branch-protection rules by default.

How `refactor-ccds` works

Never move files ad hoc — analyze before touching anything:

1. **Inventory & analyze** the repo first: classify every file, flag anything that **doesn't cleanly fit** the standard structure or would need an actual behavior change to move.
2. **Grill session** on that ambiguous set — resolve every open question with the user before moving a single file.
3. **Prepare a clear plan**: full old → new path mapping, deletions, and behavior changes called out explicitly.
4. **Present for approval** — wait for explicit confirmation before touching the filesystem.
5. **Then refactor**: move on a new branch with `git mv` (history preserved), fix what broke, check tests against baseline.

Future steps

- Right now, `CLAUDE-CCDS.md` bakes in **one fixed** set of style choices (Tyro vs. typer, `Protocol` decomposition, wandb, ...) — the same for every project and every user.
- **Make it customizable**: turn these choices into preferences instead of hardcoded defaults, so different projects/collaborators can deviate where it makes sense.
- Concretely: a **skill that interviews the user** about their preferred style (naming, tooling choices, how strict to be about `Protocols`, testing rigor, ...), then **regenerates/updates** the project's `CLAUDE.md` conventions from those answers — rather than everyone hand-editing the same fixed file.